

halfedge: BVH 加速结构模块设计文档

src/bvh.rs

halfedge 库

2026-07-05

目录

1	模块定位	2
2	数据结构	2
3	构建算法	3
3.1	整体流程	3
3.2	递归分裂	3
3.3	退化守卫	4
3.4	复杂度	4
4	射线-AABB 求交: Slab 法	4
4.1	公式	4
4.2	平行 slab 处理	5
4.3	返回值	5
5	射线-网格求交 (BVH 加速)	5
5.1	递归算法	5
5.2	剪枝条件	5
5.3	子节点访问顺序	6
5.4	复杂度	6
6	最近点查询 (BVH 加速)	6
6.1	点-AABB 最近距离平方	6
6.2	递归算法	6
6.3	剪枝条件	6
6.4	复杂度	6
7	与暴力算法的关系	7
8	退化守卫汇总	7
9	测试覆盖	8

1 模块定位

bvh.rs 在 geometry.rs 之上叠加**空间加速结构**能力，将射线求交与最近点查询从 $O(F)$ 暴力扫描降到平均 $O(\log F)$ 。对大网格 ($> 10^3$ 面) 有数量级加速，对小网格 (< 64 面) BVH 自身常数开销可能让它略慢于暴力。

核心 API:

类别	API	语义
构建	<code>Bvh::new()</code>	创建空 BVH
	<code>Bvh::build(&mesh)</code>	从网格构建 (仅三角面), 返回 <code>Bvh</code>
查询	<code>bvh.ray_intersection(o, d, &mesh)</code>	射线求最近交点, 返回 <code>Option<RayHit></code>
	<code>bvh.nearest_point(p, &mesh)</code>	网格表面最近点, 返回 <code>(FaceId, [f64;3], f64)</code>
	<code>bvh.faces_in_aabb(&AABB, &mesh)</code>	返回 AABB 重叠的候选面 (布尔运算用)
	<code>bvh.ray_count_hits(o, d, &mesh)</code>	射线命中三角面总数 (射线奇偶法用)
统计	<code>bvh.node_count() / leaf_count() / depth()</code>	节点数 / 叶子数 / 树深
	<code>bvh.is_empty()</code>	是否无三角面
	<code>LEAF_MAX_FACES</code> (常量 = 8)	叶子节点最大面数

为布尔运算新增的查询 `faces_in_aabb` 与 `ray_count_hits` 专为 `boolean.rs` 的 `corefinement` 加速而新增:

- `faces_in_aabb`: 递归遍历 BVH, 节点 AABB 与查询 AABB 不重叠则剪枝; 叶子节点内逐面计算 AABB 精筛。返回的只是 候选集, 调用方仍需 `segment_triangle_intersection` 做精确判定。复杂度 $O(\log F + k)$, k 为候选面数。
- `ray_count_hits`: 与 `ray_intersection` 不同的是**统计所有命中**而非返回最近交点, 因为射线奇偶法需要交点总数的奇偶性。复杂度 $O(\log F + k)$ 。

2 数据结构

BVH 是一棵二叉树, 节点统一表示为 `BvhNode`:

```
1 struct BvhNode {
2     aabb: AABB,           //
3     left: Option<usize>,  //
4     right: Option<usize>, //
5     faces: Vec<FaceId>,   //          ID
6 }
```

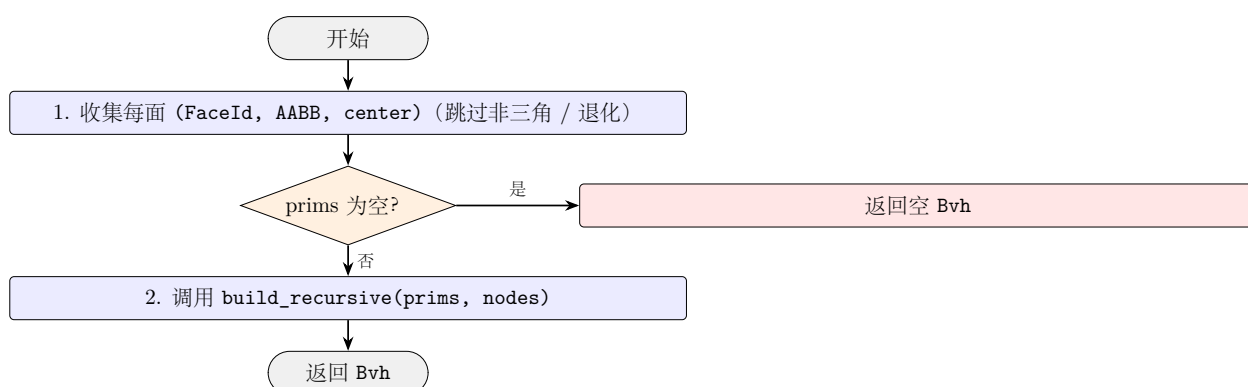
- 内部节点: `left` 与 `right` 都为 `Some`, `faces` 为空;

- 叶子节点: `left` 与 `right` 都为 `None`, `faces` 非空;
- `is_leaf()` 通过 `left.is_none() && right.is_none()` 判定。

Bvh 持有 `nodes: Vec<BvhNode>` 与 `root: Option<usize>`, 全部节点存储在连续 `Vec` 中 (cache-friendly)。

3 构建算法

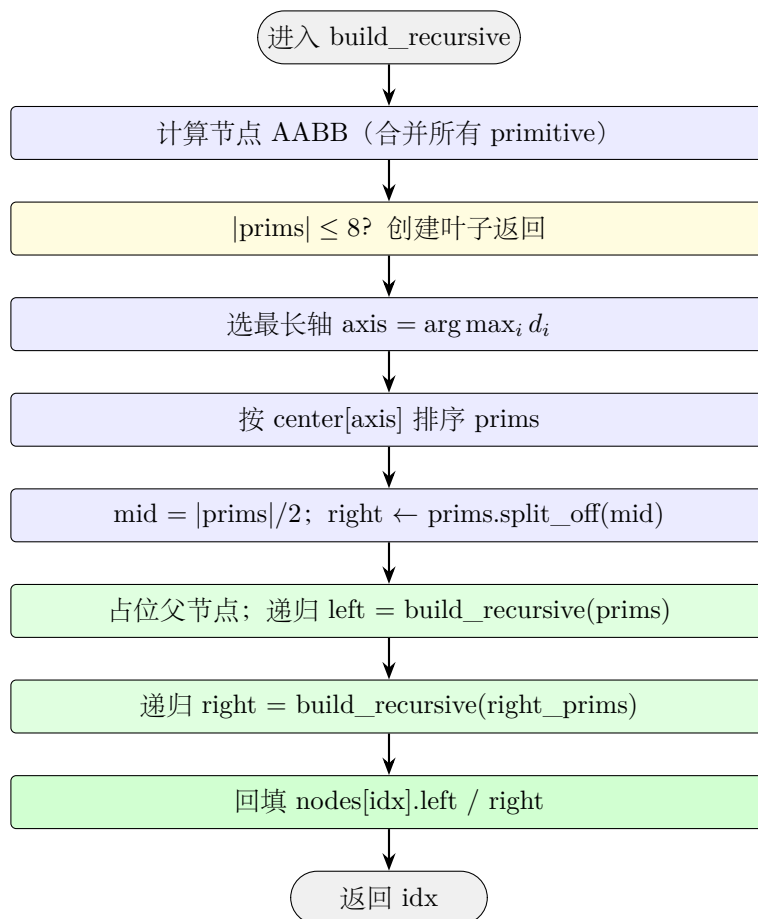
3.1 整体流程



3.2 递归分裂

`build_recursive` 对当前 `prims` 列表:

1. 计算 **节点 AABB**: 所有 primitive AABB 的合并;
2. 终止条件: 若 $|\text{prims}| \leq \text{LEAF_MAX_FACES} = 8$, 创建叶子节点 (保存面 ID 列表) 并返回;
3. 选择最长轴: 计算 AABB 对角线 $\vec{d}$, 取 $\text{axis} = \arg \max_i d_i$;
4. 按面中心排序: `prims.sort_by` 按 `center[axis]` 升序;
5. 中位数划分: `mid = prims.len() / 2`, `right_prims = prims.split_off(mid)`;
6. 占位父节点: 先 `push` 父节点 (`left/right = None`), 再递归构建左右子树, 最后回填索引。



3.3 退化守卫

构建时跳过：

- 非三角面（边界环长度 $\neq 3$ ）；
- 顶点缺失的面；
- 退化三角形（AABB 对角线 $< 10^{-14}$ ，含重复顶点或共线顶点）。

3.4 复杂度

- 时间： $O(F \log F)$ （每层 $O(F)$ 排序，共 $\log F$ 层）；
- 空间： $O(F)$ （节点数 $\leq 2F - 1$ ，满二叉树上限）；
- 树深：平均 $O(\log F)$ ，最坏 $O(F)$ （极端退化输入）。

4 射线-AABB 求交：Slab 法

4.1 公式

AABB 由 **min** / **max** 三对值定义。对每个轴 $i \in \{x, y, z\}$ ，slab 范围是 $[\min_i, \max_i]$ 。射线 $\vec{R}(t) = \vec{O} + t\vec{D}$ 在轴 i 上的进入/退出参数：

$$t_i^{\min} = \frac{\min_i - O_i}{D_i}, \quad t_i^{\max} = \frac{\max_i - O_i}{D_i},$$

若 $t_i^{\min} > t_i^{\max}$ 互换。

相交条件：

$$t_{\text{enter}} = \max_i \min(t_i^{\min}, t_i^{\max}), \quad t_{\text{exit}} = \min_i \max(t_i^{\min}, t_i^{\max}),$$

$$t_{\text{enter}} \leq t_{\text{exit}} \quad \wedge \quad t_{\text{exit}} \geq 0.$$

4.2 平行 slab 处理

若 $|D_i| < 10^{-14}$ (射线与 slab 平行)，则需 $O_i \in [\min_i, \max_i]$ ，否则不相交。

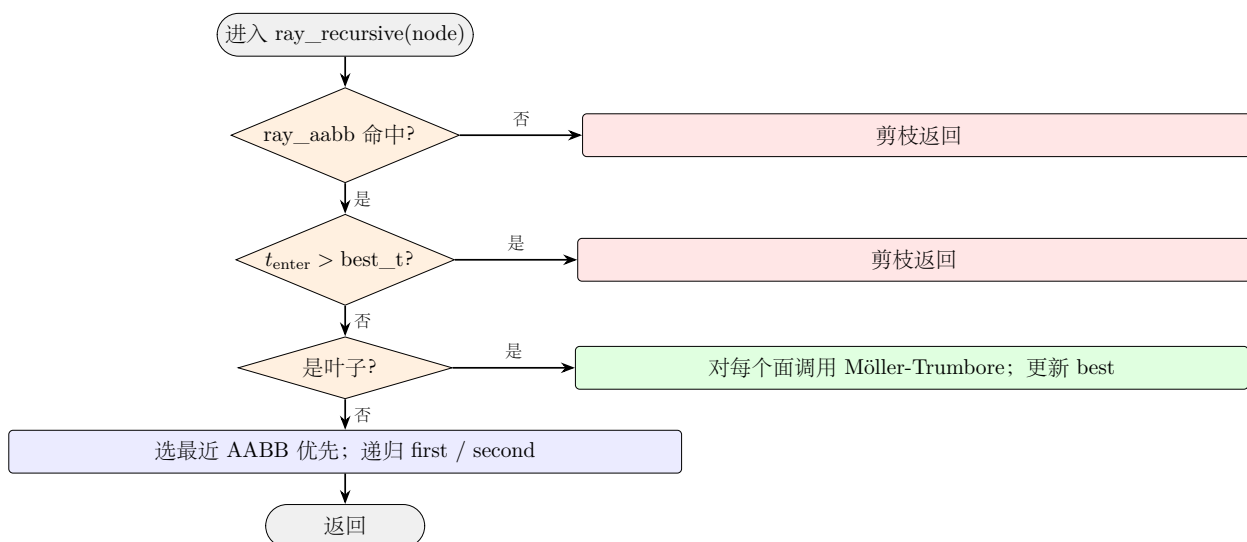
4.3 返回值

返回 `Option<(t_enter, t_exit)>`：

- `t_enter`：射线进入 AABB 的参数 (可为负，原点在 AABB 内时)；
- `t_exit`：射线离开 AABB 的参数。

5 射线-网格求交 (BVH 加速)

5.1 递归算法



5.2 剪枝条件

1. **AABB 剪枝**：射线与节点 AABB 不相交 $\rightarrow$ 整个子树跳过；
2. **距离剪枝**：当前已知最近交点 `best_t` 小于 AABB 进入参数 `t_enter` $\rightarrow$ 子树不可能产生更近交点；
3. **叶子求交**：对叶子节点内的每个面调用 `ray_triangle_intersection` (Möller-Trumbore)，更新 `best` 为 t 最小者。

5.3 子节点访问顺序

内部节点的两个子节点都先计算 `ray_aabb` 的 `t_enter`，优先访问 `t` 较小者。这样后续子树访问时 `best_t` 已被更新为更紧的上界，剪枝效率更高。

5.4 复杂度

平均 $O(\log F)$ ；最坏 $O(F)$ （射线擦过大量 AABB）。

6 最近点查询（BVH 加速）

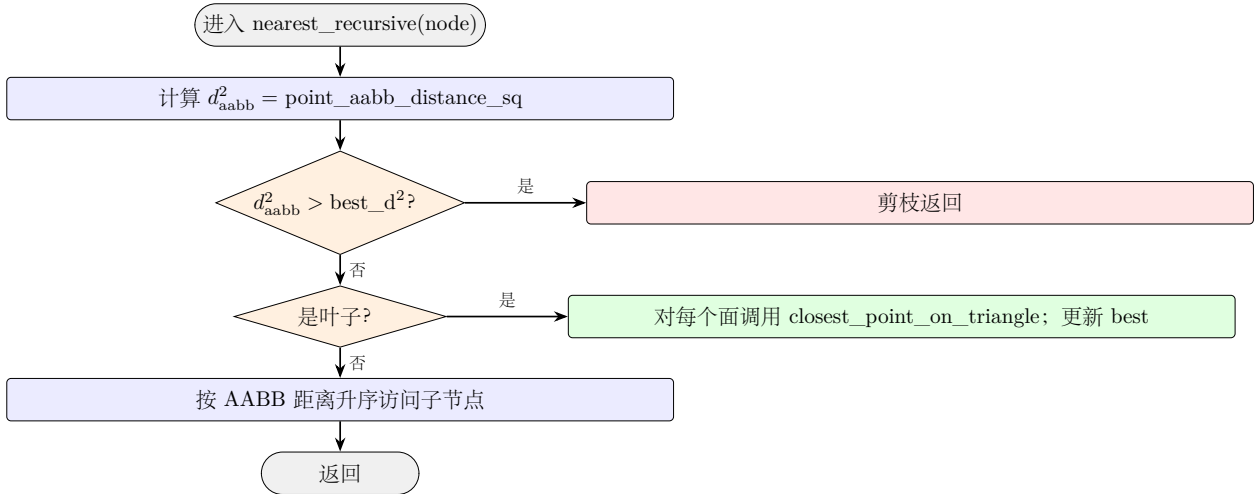
6.1 点-AABB 最近距离平方

对查询点 $\vec{P}$ 与 AABB：

$$d^2(\vec{P}, \text{AABB}) = \sum_i \max(0, \min_i - P_i)^2 + \max(0, P_i - \max_i)^2.$$

点在 AABB 内时距离为 0。

6.2 递归算法



6.3 剪枝条件

AABB 距离剪枝：若节点 AABB 的最近距离平方 d_{aabb}^2 大于当前已知最近距离平方 best_d^2 ，则该子树不可能产生更近的交点 → 剪枝。

子节点访问顺序：内部节点的两个子节点都先计算 `point_aabb_distance_sq`，按距离升序访问。先访问近的子树，使 `best_d` 更紧，剪枝更高效。

6.4 复杂度

平均 $O(\log F)$ ；最坏 $O(F)$ （查询点周围面密度极高）。

7 与暴力算法的关系

操作	暴力 (geometry.rs)	BVH (bvh.rs)	加速比 (大网格)
射线求交	<code>ray_mesh_intersection</code> $O(F)$	<code>bvh.ray_intersection</code> $O(\log F)$	$\sim F / \log F$
最近点	遍历面 + <code>point_triangle_distance</code> $O(F)$	<code>bvh.nearest_point</code> $O(\log F)$	$\sim F / \log F$

何时用 BVH:

- 网格 $> 10^3$ 面, 且需多次射线 / 最近点查询;
- BVH 构建开销 $O(F \log F)$, 单次查询摊销下值得。

何时用暴力:

- 网格 < 64 面, BVH 常数开销不划算;
- 仅需一次或几次查询, 构建开销无法摊销。

8 退化守卫汇总

函数	退化场景	处理
<code>Bvh::build</code>	空网格 / 无三角面	返回空 <code>Bvh</code> (<code>root = None</code>)
<code>Bvh::build</code>	退化三角形 (共线 / 重复顶点)	跳过该面
<code>ray_intersection</code>	空 BVH	返回 <code>None</code>
<code>nearest_point</code>	空 BVH	返回 <code>None</code>
<code>ray_aabb</code>	射线与 slab 平行	检查原点是否在 slab 内
<code>ray_aabb</code>	AABB 在射线反方向	<code>t_exit < 0</code> 返回 <code>None</code>
<code>point_aabb_distance_sq</code>	点在 AABB 内	返回 0

9 测试覆盖

测试名	验证点
构建	
build_empty_mesh	空网格 BVH 为空
build_two_triangles_basic	2 面 $\leq 8 \rightarrow$ 单叶子节点
large_mesh_bvh_depth_is_logarithmic	icosphere(3) 80 面 BVH 深度 ≤ 12
射线求交	
ray_hit_first_triangle	射线命中第一个三角形
ray_hit_second_triangle_when_first_missed	射线只命中第二个三角形
ray_miss_returns_none	远离网格的射线返回 <code>None</code>
ray_returns_nearest_hit	icosphere 球面命中半径 ≈ 1
icosphere_bvh_consistent_with_brute_force_ray	BVH 与暴力算法 t 一致
最近点	
nearest_point_on_triangle_returns_vertex	远点最近点为顶点
nearest_point_inside_triangle	三角形上方点投影到面
nearest_point_far_away_picks_closer_triangle	选择更近的三角形
icosphere_bvh_consistent_with_brute_force_nearest	BVH 与暴力最近距离一致
辅助函数	
ray_aabb_basic_hit	射线-AABB 命中 / 未命中
ray_aabb_parallel_slab	平行 slab 边界条件
point_aabb_distance_basic	点-AABB 距离（内部 / 角落 / 面外）

src/bvh.rs 共 15 个单元测试，全库共 371 个。